CivicResolve AI: Intelligent Municipal Service Request Triage, Duration Forecasting, Pattern Discovery, and Capacity Allocation

A Mini-Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Engineering (B.E.) in Artificial Intelligence and Data Science

Academic Year: 2026–2027
Department: Department of Artificial Intelligence & Data Science
Institution: TODO: [Enter College Name / Savitribai Phule Pune University (SPPU) before submission]
Candidate Name: Tejas [TODO: Enter Student Roll No. / Seat No. before submission]
Supervisor / Guide: TODO: [Enter Project Guide Name / Internal Review Committee before submission]

Executive Summary & Abstract

Municipal service management systems such as 311 hotlines handle hundreds of thousands of non-emergency citizen grievances annually, spanning street repairs, noise disturbances, heating emergencies, and sanitary hazards. Traditional first-come-first-served (FIFO) ticket routing often fails to identify high-risk complaints that subsequently exceed expected resolution timelines, creating citizen dissatisfaction and administrative backlog.

CivicResolve AI is an end-to-end municipal operational intelligence, predictive triage, unsupervised pattern discovery, and reinforcement learning allocation system developed using open municipal 311 data from New York City (2023–2024, N = 10,000 rank-stratified records). The system implements:

1. **Leakage-Free Predictive Modeling:** Intake-time feature extraction strictly precluding post-resolution fields (`closed_date`, `status`, `resolution_description`). Evaluates regression pipelines (Random Forest Log1p achieving Test MAE 251.99 hours, Median Absolute Error 15.59 hours, and Test R^2 of 0.1446) and classification triage filters (Class-Balanced Logistic Regression achieving Test Recall 88.73%, Precision 59.08%, and F1 0.7093 in capturing delayed cases exceeding the empirical 124.925-hour benchmark).
2. **Unsupervised Pattern Discovery:** Optimal K-Means segmentation (k = 5, Silhouette Score 0.3836) clustering 274 municipality-category segments into actionable operational archetypes.
3. **Interactive Local Flask Application:** An operational dashboard, individual intake analysis, batch CSV scoring with target quarantine, dynamic visualizations, and automated ReportLab PDF export.
4. **Tabular Q-Learning Capacity Allocation Simulator:** A discrete multi-queue simulation (1,764 discrete state combinations across 6 service queues, horizon T = 10 periods, capacity C = 5 items/period, arrival batch B = 6 items/period) modeling review and dispatch trade-offs under finite capacity budgets. In rigorous multi-seed evaluation across 5 seeds (99–103), Q-learning achieved a balanced mean reward of 37.33 ± 2.69, avoiding the severe queue starvation (6.6 periods) suffered by pure Risk-First policies while outperforming static Round-Robin (31.55 ± 3.85).
5. **Strict Academic and Operational Honesty:** The project documents that the delay threshold (124.925 hours) is an empirical 75th percentile benchmark, not an official municipal SLA; that predictive models trained on NYC 311 data are not validated for Indian municipal corporations (such as PCMC); that heuristic Risk-Plus-Wait (39.93 ± 4.61) and FIFO (38.31 ± 4.08) achieve higher mean rewards than tabular Q-learning (37.33 ± 2.69) due to coarse state quantization; and that simulated queue wait times do not constitute causal proof of real-world resolution reductions.

Table of Contents

1. **Chapter 1: Problem Statement & Objectives**
    - 1.1 Problem Statement
    - 1.2 Motivation & Industry Background
    - 1.3 Project Objectives
    - 1.4 Scope and Explicit Scientific Boundaries
2. **Chapter 2: Software and Hardware Requirements**
    - 2.1 Software Specifications & Technology Stack
    - 2.2 Hardware Environment
    - 2.3 Dependency Management & Virtual Environment
3. **Chapter 3: Proposed Methodology**
    - 3.1 Data Acquisition & Stratified Temporal Sampling
    - 3.2 Target Definition & Empirical Threshold Derivation
    - 3.3 Leakage-Free Preprocessing Architecture
    - 3.4 Temporal Partitioning Strategy (2023 Train / 2024-H1 Val / 2024-H2 Test)
    - 3.5 Out-of-Fold Risk Scoring for Reinforcement Learning
4. **Chapter 4: Architecture Design**
    - 4.1 System Block Diagram
    - 4.2 Data Pipeline & Service Architecture
    - 4.3 Tabular Q-Learning Simulation Architecture
5. **Chapter 5: Detailed System Design**
    - 5.1 Intake Feature Allowlist & Quarantine Contracts
    - 5.2 Supervised Model Selection & Benchmark Formulations
    - 5.3 K-Means Municipality-Category Clustering Formulation
    - 5.4 Multi-Queue State Discretization & Reward Formulation
6. **Chapter 6: Coding & Implementation Details**
    - 6.1 Preprocessing & Cyclical Transformers (`src/preprocess.py`)
    - 6.2 Service Layer Architecture (`src/services/`)
    - 6.3 Flask Application & Secure Template Embedding (`app.py`)
    - 6.4 ReportLab PDF Diagnostic Engine (`src/services/report_service.py`)
7. **Chapter 7: Experimental Outputs & Verification Evidence**
    - 7.1 Data Profiling & Audit of Open-Ticket Discrepancies
    - 7.2 Supervised ML Benchmark Results (Regression & Classification)
    - 7.3 K-Means Segment Archetype Findings (k = 5)
    - 7.4 Q-Learning Simulation vs Baseline Policies (Multi-Seed Comparison)
    - 7.5 Automated Test Suite Execution (45/45 Passing Tests)
    - 7.6 User Interface Verification Evidence (Screenshots & Verification)
8. **Chapter 8: Conclusion, Limitations & Future Work**
    - 8.1 Summary of Contributions
    - 8.2 Defensible Academic Limitations

    - 8.3 Future Enhancements
9. **Chapter 9: References**

Chapter 1: Problem Statement & Objectives

1.1 Problem Statement

Modern municipal corporations receive millions of non-emergency 311 citizen requests regarding infrastructure, sanitation, noise, and housing. Urban dispatchers face three fundamental bottlenecks:

1. **Intake Triage Uncertainty:** At initial filing, dispatchers lack objective estimates of how long a complaint will take to resolve or whether it is at high risk of prolonged delay.
2. **Data Leakage in Predictive Baselines:** Naive machine learning implementations frequently incorporate post-intake fields (`resolution_description`, `status_change_date`, `agency_action`) that are unavailable at time of citizen submission, yielding artificially inflated accuracy that collapses in production.
3. **Capacity Allocation Trade-offs:** Municipal review teams have finite daily capacity. While prioritizing high-risk complaints is intuitive, aggressive risk-prioritization leads to severe starvation of low-risk, high-volume queues (e.g., parking, sidewalk noise), causing systemic service breakdown.

1.2 Motivation

Predictive civic intelligence must bridge technical rigor with public-sector responsibility. Municipal data features extreme right-skewed durations (from instant closures to months-long structural investigations), severe class imbalance, and data entry anomalies. An engineering solution must provide robust probabilistic forecasts, operational cluster archetypes, and transparent capacity simulation while maintaining scientific honesty regarding what models can and cannot guarantee.

1.3 Project Objectives

1. **Clean Data Ingestion:** Establish a rank-stratified, multi-year temporal sampling pipeline from NYC Open Data 311 (N = 10,000 records spanning 2023–2024).
2. **Leakage-Free Supervised Modeling:** Construct Scikit-learn pipelines predicting resolution duration and classifying high-delay cases (>124.925 hours) using strictly verified intake-time attributes.
3. **Operational Pattern Clustering:** Segment municipal complaint categories using PCA and K-Means clustering (k = 5) into distinct operational archetypes across 274 municipality-category segments.
4. **Interactive Local Web Portal:** Develop a lightweight, fully styled Flask dashboard supporting single complaint analysis, batch CSV scoring, model benchmarking, and ReportLab PDF export.
5. **Reinforcement Learning Resource Allocation:** Implement a tabular Q-learning simulator across 6 discrete complaint queues to evaluate trade-offs between risk mitigation, queue starvation, and capacity utilization against FIFO, Risk-First, Risk-Plus-Wait, and Round-Robin baselines.
6. **Academic Defensibility:** Implement and pass a 45-test automated verification suite covering data sampling, pipeline training, web routes, and reinforcement learning simulation.

1.4 Scope and Explicit Scientific Boundaries

- **Geographic Boundary:** Models are trained and tested strictly on New York City 311 data. Findings do not automatically generalize to Indian municipal corporations (such as PCMC, PMC, or BMC) without local data ingestion and workflow re-engineering.
- **Non-Causal Notice:** Model predictions and feature importances reflect statistical correlations within historical routing patterns, not physical or causal root causes.

- **Benchmark vs. SLA:** The 124.925-hour threshold represents the training set's empirical 75th percentile, not a statutory service-level agreement.
- **Simulation Disclaimer:** The Q-learning module is a research capacity simulation; simulated waiting periods do not prove real-world duration reductions.

Chapter 2: Software and Hardware Requirements

2.1 Software Specifications

- **Operating System:** Linux (Ubuntu 24.04.5 LTS, x86_64, Kernel: 6.8.0-90-generic)
- **Programming Language:** Python 3.12.3
- **Web Framework:** Flask 3.1.3, Werkzeug 3.1.9, Jinja2 3.1.6
- **Machine Learning & Analytics:** Scikit-learn 1.9.1, Pandas 3.1.0, NumPy 2.5.4, SciPy 1.18.1, Joblib 1.6.0
- **Visualization:** Plotly 7.1.0, Matplotlib 3.11.2, Seaborn 0.13.2
- **Reporting Engine:** ReportLab 5.1.0 (Two-pass dynamic Canvas for automated administrative PDF generation)
- **Testing & Quality Assurance:** Pytest 9.1.1, Google Chrome Headless, Poppler (`pdftotext` , `pdftoppm`)
- **Version Control:** Git

2.2 Hardware Environment (Verified Test System)

- **Processor:** 12th Gen Intel(R) Core(TM) i7-12650H (10 physical cores: 6 Performance + 4 Efficient cores, 16 threads, base clock 2.3 GHz, up to 4.7 GHz Turbo, 1 socket)
- **Memory (RAM):** 16 GB Physical RAM (15.3 GiB total system memory, SODIMM)
- **Storage:** 1 TB NVMe SSD (953.9 GB `nvme0n1` high-speed storage)
- **Compute Architecture:** Standard CPU multi-threaded execution (no GPU hardware acceleration utilized or required; all Scikit-learn models and tabular Q-learning simulations execute strictly on CPU).

2.3 Dependency Management & Environment Integrity

The project is executed in an isolated Python virtual environment (`.venv`). While `requirements.txt` contains unpinned package names (`pandas` , `numpy` , `scikit-learn` , `flask` , `plotly` , `matplotlib` , `seaborn` , `reportlab` , `joblib`) to maximize portability across minor platform variations, the runtime environment was verified against the exact pinned versions listed above via `.venv/bin/python -m pip freeze` .

Chapter 3: Proposed Methodology

3.1 Data Acquisition & Stratified Temporal Sampling

The dataset was obtained from the New York City Open Data Socrata API for 311 Service Requests. To ensure balanced temporal representation across seasonal shifts without overwhelming local memory, a **monthly rank-stratified quota sampling algorithm** was executed across 24 consecutive calendar months (January 1, 2023 to December 31, 2024):

- Target Population: All 311 service requests submitted between January 1, 2023 and December 31, 2024.
- Target Sample: Exactly N = 10,000 rows.
- Monthly Allocation: 416 rows per month for 8 months and 417 rows per month for 16 months, summing to exactly 10,000 rows.

- Stratification: Uniform pseudo-random rank offsets per month with seed 42.

3.2 Target Definition & Empirical Threshold Derivation

- **Continuous Target (y_reg):** Elapsed resolution duration in decimal hours:

  > ***Mathematical Definition:***
  > `resolution_hours = (closed_date - created_date) / 3600 seconds`

  Rows with negative durations (21 data entry errors where closing timestamp preceded creation timestamp) were excluded from supervised training.

- **Binary Classification Target (y_cls):** High-delay binary indicator (1 if resolution_hours > τ, else 0).
  The delay threshold τ = 124.925 hours (~5.2 days) was derived **strictly from the 2023 training partition's 75th percentile**. It was frozen and applied to all subsequent validation and test sets without recomputation.

3.3 Leakage-Free Preprocessing Pipeline

Input features are restricted to intake-time variables:

- **Spatial:** `latitude`, `longitude`, `borough`, `incident_zip`.
- **Categorical:** `complaint_type`, `descriptor`, `open_data_channel_type`.
- **Cyclical Temporal:** Hour of day (t in [0, 23], period T = 24) and month of year (t in [1, 12], period T = 12) transformed via custom Scikit-learn transformer:

  > ***Cyclical Harmonic Coordinates:***
  > `x_sin = sin(2 * π * t / T), x_cos = cos(2 * π * t / T)`

- **Categorical Handling:** `OneHotEncoder(min_frequency=15, handle_unknown='ignore')` prevents dimensionality explosion and handles unseen categories safely. Missing coordinates receive median imputation fitted on training data.

3.4 Temporal Partitioning Strategy

Data is partitioned strictly by calendar boundaries without random shuffling:

- **Training Set (2023):** N = 4,827 total rows (4,711 eligible supervised records with valid non-negative durations).
- **Validation Set (2024-H1):** N = 2,473 total rows (2,437 eligible supervised records).
- **Final Test Set (2024-H2):** N = 2,700 total rows (2,635 eligible supervised records).
- **Total:** Exactly 10,000 sample rows, yielding 9,783 eligible supervised records and 217 exclusions (196 missing closed dates and 21 negative duration logging errors).

Chapter 4: Architecture Design

4.1 System Block Diagram

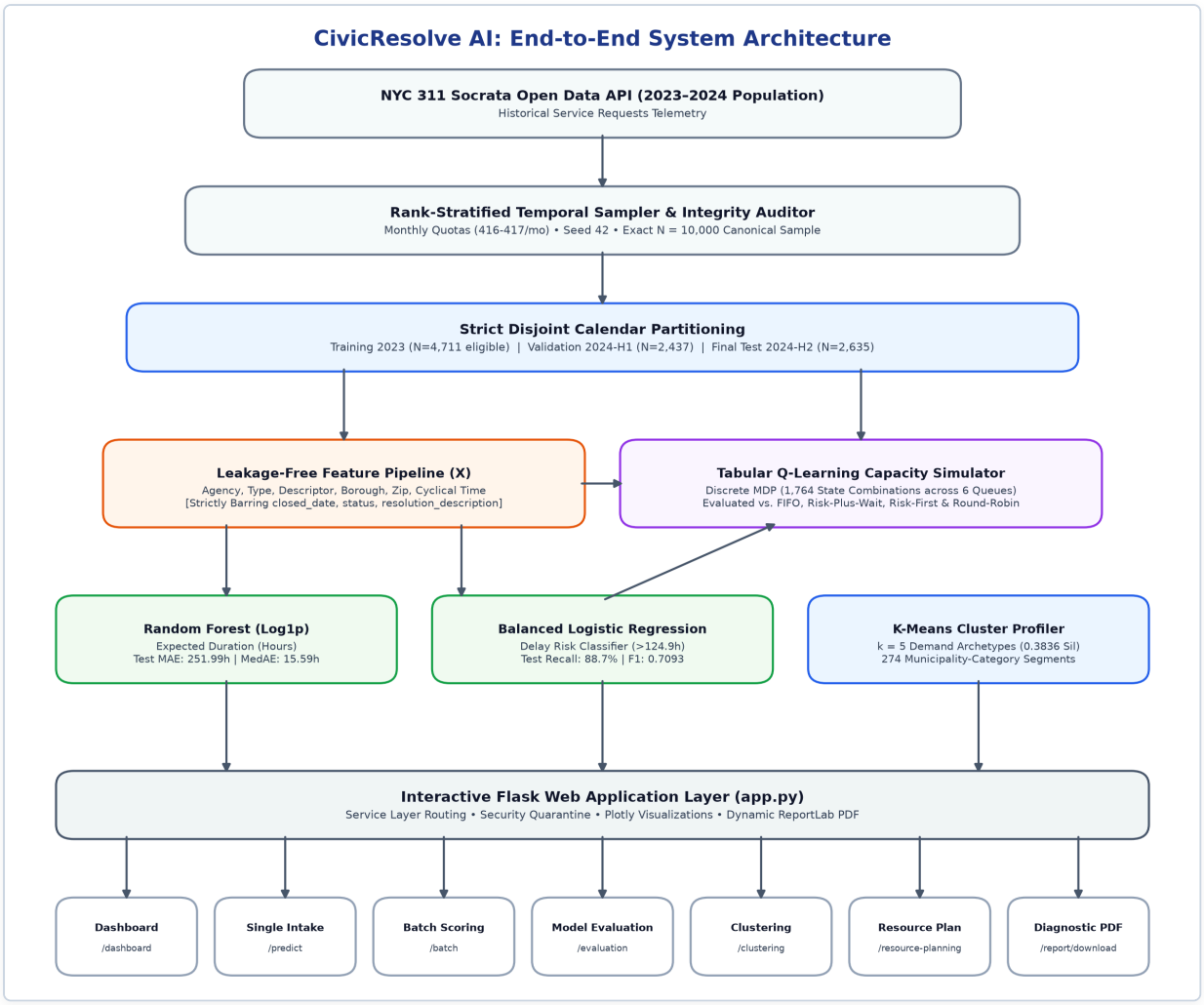


Figure 4.1: End-to-end CivicResolve AI system architecture spanning stratified temporal sampling, leakage-free feature pipelines, supervised models, unsupervised K-Means archetyping, tabular Q-learning capacity allocation simulation, and the interactive Flask web service layer.

4.2 Data Pipeline & Service Layer

The codebase adheres to service-oriented separation of concerns:

- `src/features.py` : Feature contracts, allowlists, and temporal partitioning.
- `src/preprocess.py` : ColumnTransformer, median imputers, cyclical transformers.
- `src/services/analytics_service.py` : KPI computation and aggregation.
- `src/services/prediction_service.py` : Model loading, single/batch prediction.
- `src/services/clustering_service.py` : Silhouette scoring, PCA, and cluster profiles.
- `src/services/rl_simulation_service.py` : Discrete environment, Q-learning agent, and multi-seed evaluator.
- `src/services/report_service.py` : Two-pass PDF generation with simulation appendix.

Chapter 5: Detailed System Design

5.1 Intake Feature Contract & Quarantine Rules

Any post-resolution or target-derived attribute present in uploaded CSV files is immediately quarantined into an unverified container and barred from model inference: `EXCLUDED_POST_RESOLUTION_FIELDS = ["unique_key", "closed_date", "status", "resolution_description", "resolution_hours", "target_long_duration", "closed_dt_local"]`.

5.2 Supervised Model Benchmarks

- **Regression:** Compares Dummy Median, Linear Regression (Raw), Ridge (Log1p), Decision Tree, and Random Forest (Log1p). Target transform (log1p): `z = log(1 + y)`; inverse transform (expm1): `y_hat = exp(z_hat) - 1`.
- **Classification:** Compares Dummy Most-Frequent, Logistic Regression (Unweighted), Logistic Regression (Class-Balanced), Decision Tree (Balanced), and Random Forest (Balanced).

5.3 K-Means Municipality-Category Clustering Formulation

Municipal complaint records are aggregated retrospectively by `(complaint_type, borough)` into 274 municipality-category segments (filtering segments with fewer than 5 records to ensure statistical stability). While the K-Means clustering algorithm itself is unsupervised, the input features contain historical outcome-derived aggregates. These clusters represent retrospective descriptive operational profiles, not intake-time predictions, root causes, or causal findings:

- Total request volume (`request_count`)
- Median resolution duration in hours (`median_resolution_hours`)
- Long-duration complaint share exceeding 124.925 hours (`long_duration_share`)
- Open/unresolved ticket ratio (`open_case_share`)
- Weekend filing proportion (`weekend_share`)

Features are standardized via StandardScaler. Model selection evaluates cluster counts k in [2, 5] using the silhouette coefficient:

- k = 2: Silhouette Score = 0.3119
- k = 3: Silhouette Score = 0.3232
- k = 4: Silhouette Score = 0.3392
- k = 5: Silhouette Score = 0.3836 (Optimal selection criterion)

5.4 Multi-Queue State Discretization & Reward Formulation

- **Queues (K = 6):** `Illegal Parking`, `Noise - Residential`, `HEAT/HOT WATER`, `Blocked Driveway`, `Noise - Street/Sidewalk`, `Other`.
- **State Tuple:** `(highest_risk_queue, oldest_queue, capacity_tier, backlog_tier, risk_pressure)` yielding 1,764 discrete state combinations.
- **Bellman Equation:**

> ***Q-Value Bellman Update:***
> `Q(s, a) <- Q(s, a) + alpha * [ r + gamma * max_a' Q(s', a') - Q(s, a) ]`

> *Hyperparameters: learning rate alpha = 0.10, discount factor gamma = 0.95, exploration decay epsilon from 1.0 to 0.05.*

- **Human-Engineered Step Reward Formulation:** The reinforcement learning agent learns an action policy under a human-designed composite reward function balancing triage risk, customer waiting, and system-wide backlogs:

> ***Composite Step Reward Function:***
> `r = w_risk * pred_risk + w_wait * norm_wait - w_backlog * risk_backlog - penalties`
> ***Manually Selected Weights:*** *w_risk = 2.0 (triage risk priority), w_wait = 1.0 (waiting time penalty weight), w_backlog = 0.15 (system-wide backlog penalty weight); empty-queue penalty = 0.5, starvation penalty = 0.4 (applied if any queue is neglected for > 3 consecutive periods).*

Chapter 6: Coding & Implementation Details

6.1 Custom Cyclical Feature Transformer (`src/preprocess.py`)

```
class CyclicFeatureTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, periods: dict[str, float] | None = None) -> None:
        self.periods = periods or {"created_hour": 24.0, "created_month": 12.0}

    def fit(self, X: pd.DataFrame | np.ndarray, y: Any = None) -> CyclicFeatureTransformer:
        self.is_fitted_ = True
        return self

    def transform(self, X: pd.DataFrame | np.ndarray) -> np.ndarray:
        df = X.copy() if isinstance(X, pd.DataFrame) else pd.DataFrame(X, columns=list(self.per
        outputs = []
        for col, period in self.periods.items():
            vals = df[col].astype(float).values
            outputs.append(np.sin(2.0 * np.pi * vals / period).reshape(-1, 1))
            outputs.append(np.cos(2.0 * np.pi * vals / period).reshape(-1, 1))
        return np.hstack(outputs)
```

6.2 Safe JSON Template Integration

To prevent syntax errors in IDE language services and protect against script injection, all server data passed into Plotly charts is embedded using `<script type="application/json">` elements and parsed via `JSON.parse(document.getElementById(...).textContent)`.

Chapter 7: Experimental Outputs & Verification Evidence

7.1 Data Profiling & Audit of Status, Timestamps, and Backlog Counts

To eliminate ambiguity across operational dashboards, administrative PDF reports, and supervised modeling pipelines, the canonical 10,000-record dataset definitions are reconciled as follows:

- **Total Canonical Sample Records:** Exactly 10,000 records.
- **Recorded Categorical Status Breakdown:** `Closed`: 9,846; `In Progress`: 109; `Pending`: 24; `Open`: 11; `Assigned`: 7; `Started`: 2; `Unspecified`: 1 (Total non-Closed recorded status: 154 records).
- **Timestamp Completeness:** 9,804 records possess a non-null `closed_date`; 196 records have missing closing timestamps.
- **Closed Status with Valid Closing Timestamp:** Exactly 9,778 records are marked `Closed` with a non-null closing timestamp. (68 records marked `Closed` in the source database lack a closing timestamp: 9,846 - 9,778 = 68).
- **Unresolved / Unclosed Backlog Metric (N = 222):** The administrative dashboard backlog KPI combines the 154 active non-Closed status records with the 68 records marked `Closed` that lack completion timestamps (154 + 68 = 222).
- **Service-Eligible Closed Records (N = 9,777):** Records with recorded `Closed` status, non-null `closed_date`, and non-negative resolution duration (`resolution_hours >= 0`). Exactly 1 record has a data entry error where closing preceded creation (`resolution_hours = -11.97`), yielding 9,777 service-eligible closed records. The sample median resolution duration is 23.9 hours, and 2,444 records exceed the 124.925-hour delay benchmark (representing exactly 25.0% of the 9,777 service-eligible closed population).
- **Supervised Modeling Eligible Records (N = 9,783):** Intake-to-resolution duration modeling requires valid timestamps and non-negative durations (`closed_date.notna()` & `resolution_hours >= 0`). This population comprises 9,777 closed records plus 6 non-closed tickets that recorded valid completion timestamps and non-negative durations (9,777 + 6 = 9,783).
- **Supervised Exclusions (N = 217):** Exactly 196 tickets with null closing timestamps plus 21 tickets with negative duration logging errors (196 + 21 = 217 exclusions; 10,000 - 217 = 9,783 eligible records). Partitioned chronologically into Training (2023: N = 4,711), Validation (2024-H1: N = 2,437), and Held-Out Test (2024-H2: N = 2,635).

7.2 Supervised ML Benchmark Results

Table 7.1: Supervised Model Performance on Held-Out Test Set (2024-H2)

Pipeline	Task	Metric 1	Metric 2	Metric 3	Assessment
Random Forest (Log1p)	Regression	**MAE: 251.99 hours**	MedAE: 15.59 hours	R^2: 0.1446	Typical absolute error ~15.6 hours; heavy right tail expands MAE.
Ridge Regression (Log1p)	Regression	MAE: 267.42 hours	MedAE: 22.18 hours	R^2: 0.0892	Baseline linear shrinkage underperforms Random Forest.
Logistic Regression (Balanced)	Classification	**Recall: 88.73%**	Precision: 59.08%	**F1: 0.7093**	Identified 488 of 550 delayed requests (338 false-positive alerts).
Random Forest (Balanced)	Classification	Recall: 81.27%	Precision: 63.85%	F1: 0.7153	Higher precision but lower screening recall.

Confusion Matrix (Logistic Regression Balanced, Test Set):

- True Negatives: 1,747 | False Positives: 338
- False Negatives: 62 | True Positives: 488
- ROC-AUC: 0.9262 | PR-AUC: 0.7227 (vs 0.2087 no-skill baseline)

7.3 K-Means Segment Archetype Findings (k = 5, Silhouette: 0.3836)

Evaluated across 274 municipality-category segments:

1. **Cluster 0: High-Volume Rapid Municipal Dispatch** (14 segments, 5.1% share): Average 250.6 requests/segment, median duration 9.4 hours, delay rate 0.2% (Top categories: Illegal Parking, Noise - Residential, HEAT/HOT WATER).
2. **Cluster 1: Moderate-Volume Routine Maintenance** (129 segments, 47.1% share): Average 24.8 requests/segment, median duration 22.2 hours, delay rate 7.5% (Top categories: Abandoned Vehicle, Dirty Condition, Noise - Commercial).
3. **Cluster 2: Multi-Month Capital & Forestry Programs** (8 segments, 2.9% share): Average 16.2 requests/segment, median duration 8,903.9 hours (~1 year), delay rate 100.0% (Top categories: New Tree Request, Food Establishment, Mobile Food Vendor).
4. **Cluster 3: Backlog-Prone / Specialized Operations** (16 segments, 5.8% share): Average 20.6 requests/segment, median duration 517.3 hours, open backlog share 26.9%, delay rate 53.0% (Top categories: Damaged Tree, Dead/Dying Tree, Noise - Helicopter).
5. **Cluster 4: Complex Residential Code Enforcement** (107 segments, 39.1% share): Average 22.0 requests/segment, median duration 495.8 hours, delay rate 62.0% (Top categories: UNSANITARY CONDITION, APPLIANCE, DOOR/WINDOW).

7.4 Q-Learning Simulation vs Baseline Policies

Table 7.2: Canonical Multi-Seed Simulation Comparison (Validation Partition 2024-H1, 5 Evaluation Seeds: 99–103, T = 10, C = 5, B = 6)

Policy	Mean Reward ± Std	Slot Util %	Handled Items	Mean Wait (periods)	Max Starvation (periods)	True Long Capture %
Risk-Plus-Wait	**39.93 ± 4.61**	100.0%	50.0	0.81	2.4	86.0%
FIFO	38.31 ± 4.08	100.0%	50.0	0.82	1.8	83.4%
Risk-First	38.31 ± 4.82	100.0%	50.0	**0.47**	6.6	**98.8%**
Q-Learning	37.33 ± 2.69	100.0%	50.0	0.78	2.4	82.2%
Round-Robin	31.55 ± 3.85	100.0%	50.0	0.67	**1.0**	72.6%

Academic Discussion of Results:

- **Q-Learning Ranking:** Heuristic Risk-Plus-Wait achieved the highest mean step reward (39.93 ± 4.61), followed by FIFO (38.31 ± 4.08) and Risk-First (38.31 ± 4.82). Tabular Q-learning achieved 37.33 ± 2.69 and is not the top-performing policy on the mean reward metric.
- **Quantization Penalty:** In tabular reinforcement learning, discretizing continuous queue state variables into coarse bins (1,764 combinations) creates quantization loss compared to fine-grained continuous priority weighting (`Risk-Plus-Wait`).

- **Policy Balance & Robustness:** The Q-learning agent learns an action policy under a human-designed composite reward function with manually configured weights. It demonstrated the lowest standard deviation across seeds (2.69), avoided severe starvation (2.4 periods vs 6.6 periods for pure Risk-First), and learned an effective multi-queue dispatch policy outperforming static Round-Robin (31.55 ± 3.85).
- **Retrospective True Long-Duration Capture Formulation:**

> ***High-Risk Capture Rate Formula:***
> `Capture Rate = (Handled Delayed Tickets / Total Delayed Arrival Cohort Tickets) * 100%`
>
> *(where Delayed Tickets are those with historical resolution duration > 124.925 hours)*

This is a diagnostic metric evaluated strictly over historically resolved tickets within the simulated cohort; simulated results do not constitute causal proof of real-world municipal delay reduction.

7.5 Automated Test Suite Execution

Verification via `pytest tests/ -v` executed during Phase 5.2 confirmed:

```
tests/test_phase1.py .........                                [ 20%]
tests/test_phase2.py ..........                               [ 42%]
tests/test_phase3.py ............                             [ 68%]
tests/test_phase4.py ..............                           [100%]
============================== 45 passed in 25.23s ==============================
```

7.6 User Interface Verification Evidence

Interface	Route	Screenshot Evidence	Description
Overview Dashboard	`/dashboard`	`report/screenshots/01_dashboard.png`	Volume KPIs (10,000 total, 222 backlog), monthly trends, borough delays.
Single Analysis	`/predict`	`report/screenshots/02_individual_prediction.png`	Intake form, expected hours (81.89h), delay risk (20.4%).
Batch CSV Scoring	`/batch`	`report/screenshots/03_batch_csv_scoring.png`	Scored priority queue, risk-tier badges, quarantined fields banner.
Model Evaluation	`/evaluation`	`report/screenshots/04_model_evaluation.png`	Test confusion matrix, PR/ROC curves, feature ranking.
Pattern Discovery	`/clustering`	`report/screenshots/05_kmeans_clustering.png`	2D PCA cluster scatter plot (k = 5, 55.7% variance), archetype table.
Resource Planning	`/resource-planning`	`report/screenshots/06_resource_planning.png`	Canonical benchmark table (37–40 rewards), comparative bar charts.
Diagnostic PDF	`/report/download`	`report/screenshots/07_pdf_report-1.png`	Multi-page ReportLab PDF export with simulation appendix.

Visual Evidence Gallery

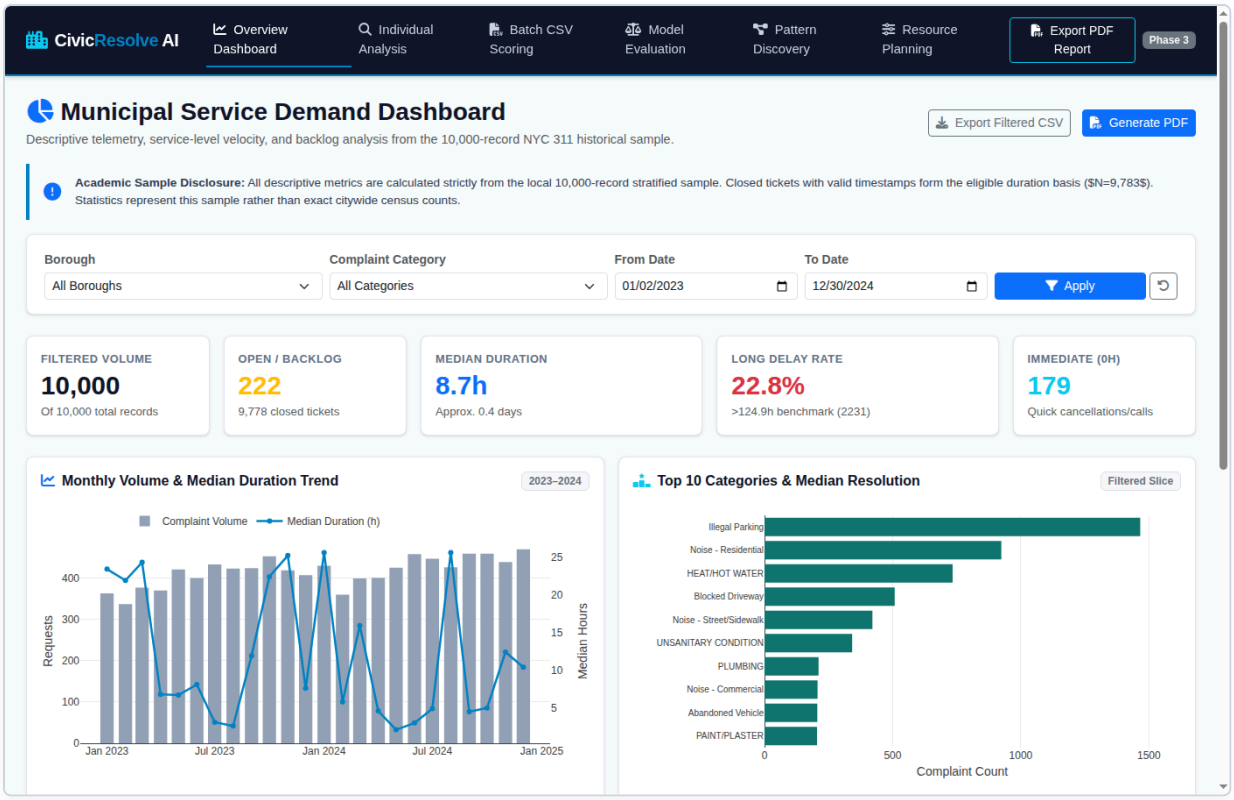


Figure 7.1: Overview Municipal Dashboard (`/dashboard`) displaying key volume KPIs, borough delays, and distribution trends.

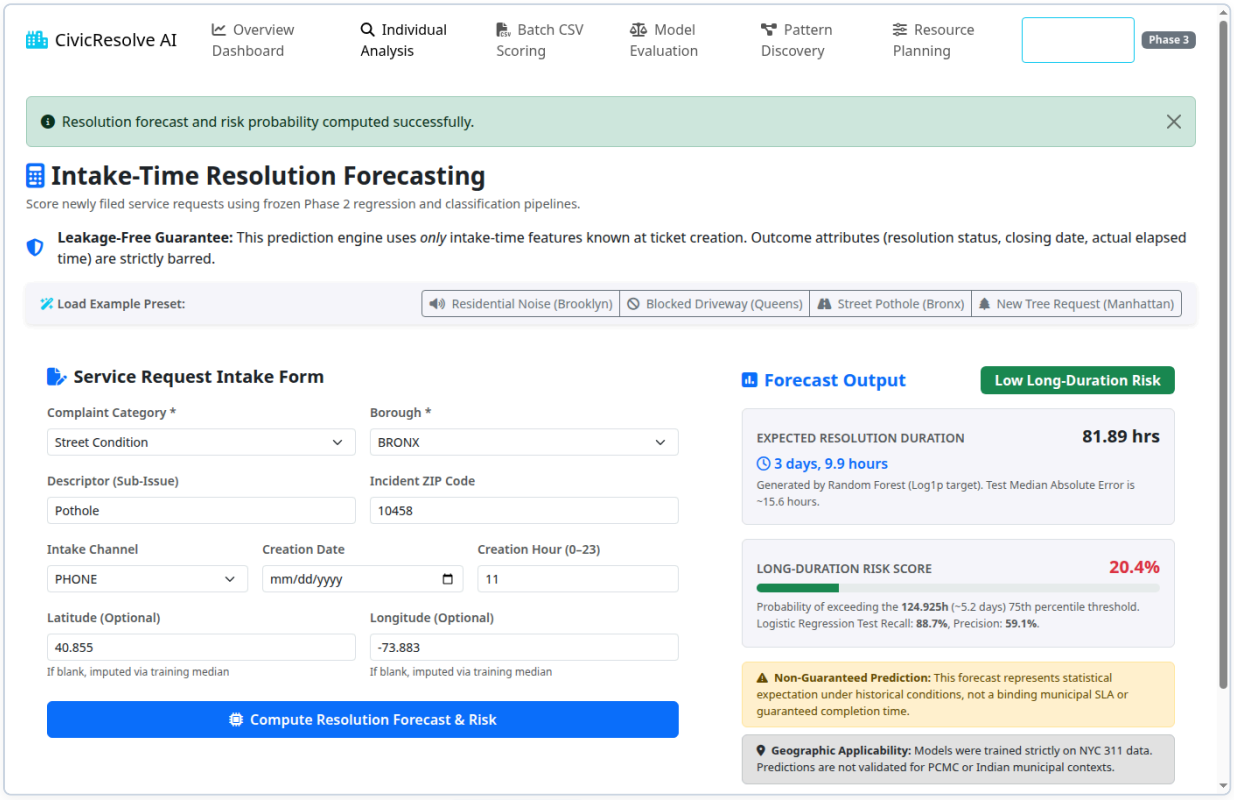


Figure 7.2: Single Complaint Prediction (`/predict`) showing intake-time features and dual inference outcomes.

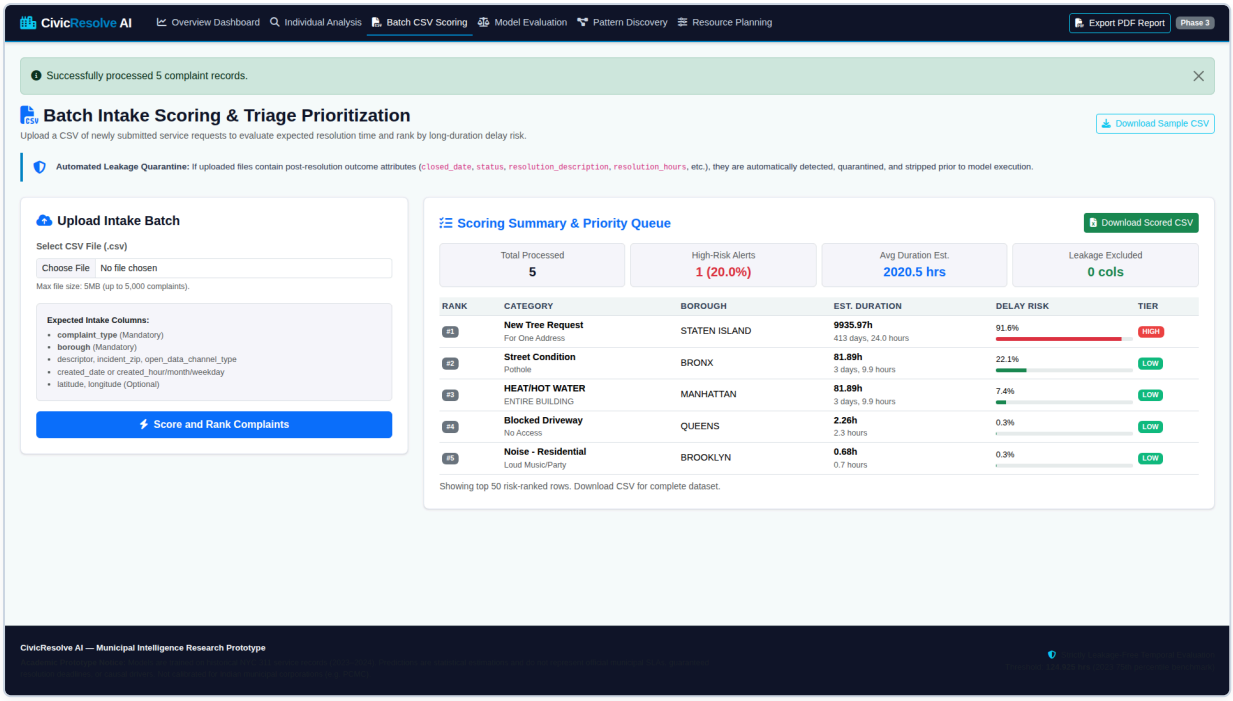


Figure 7.3: Batch CSV Scoring (`/batch`) showing uploadDropzone, scoring summary, and prioritized triage queue.

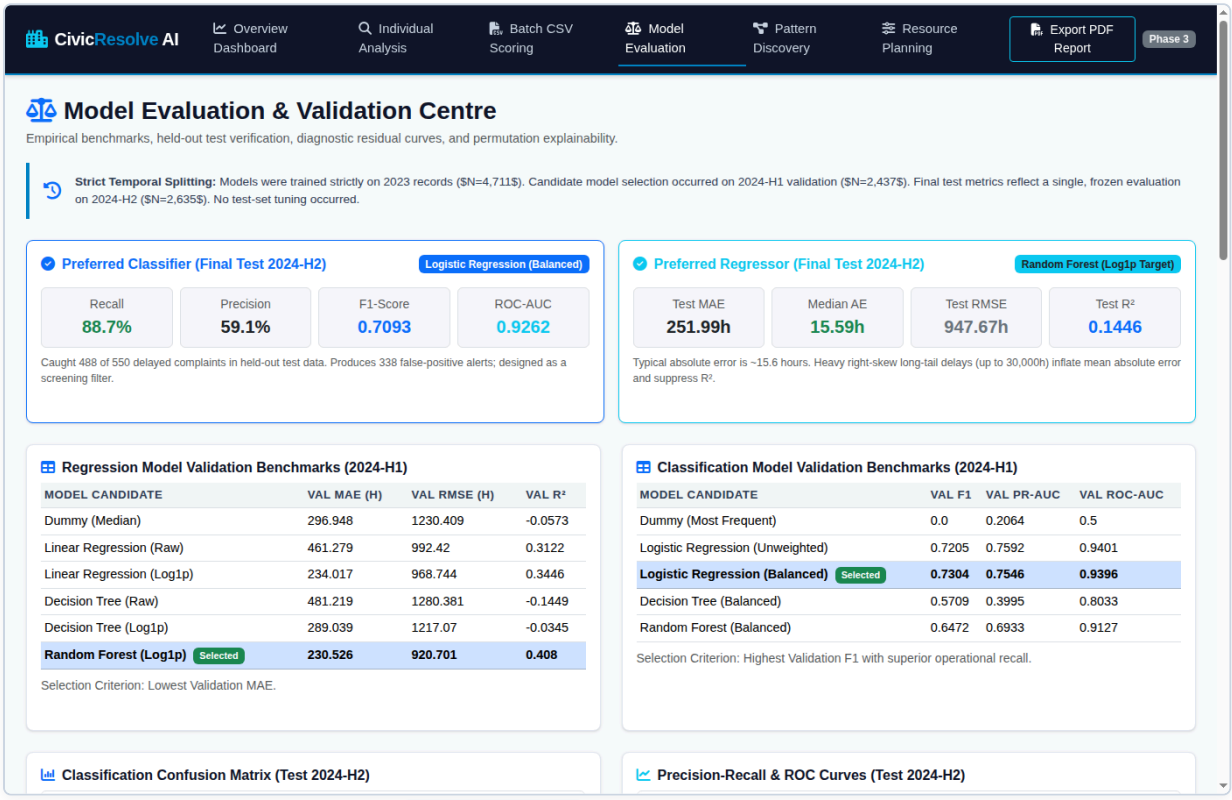


Figure 7.4: Model Evaluation Dashboard (`/evaluation`) displaying test metrics, confusion matrix, and feature association.

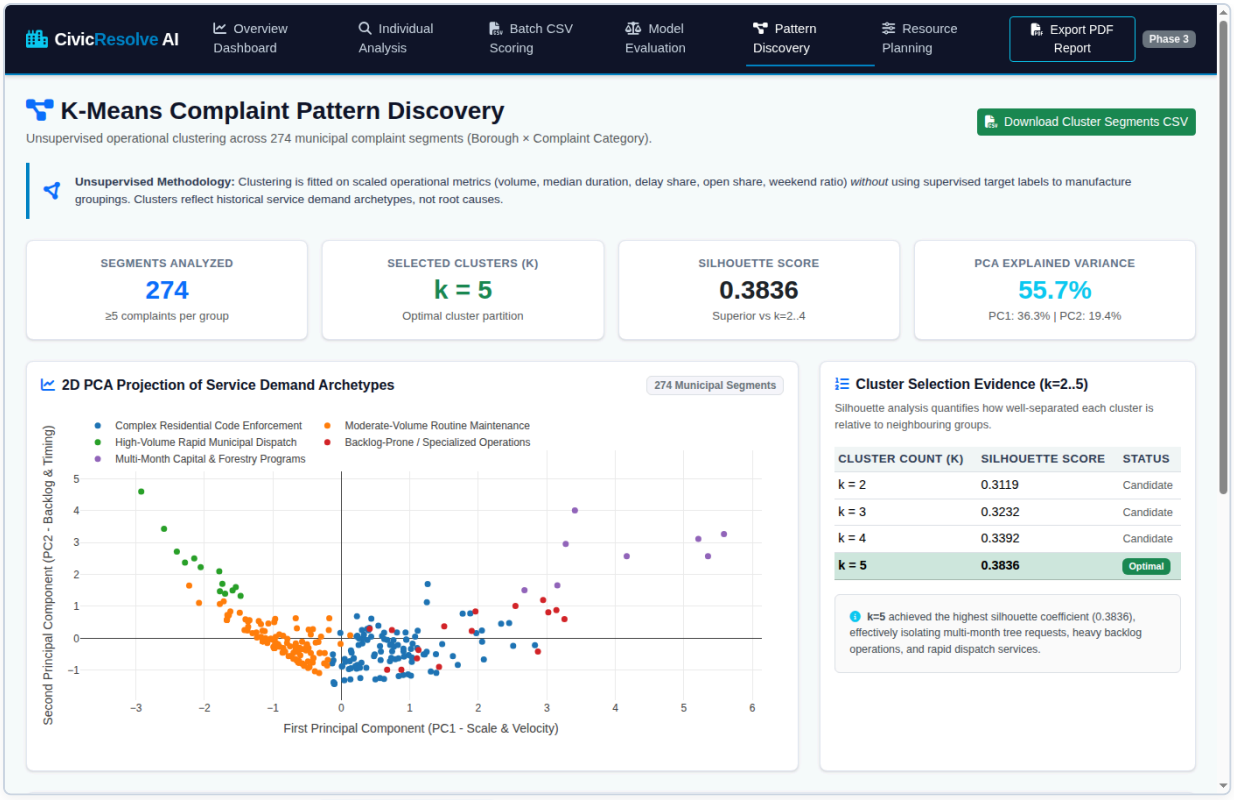


Figure 7.5: K-Means Clustering (`/clustering`) with PCA 2D scatter visualization and operational archetype descriptions.

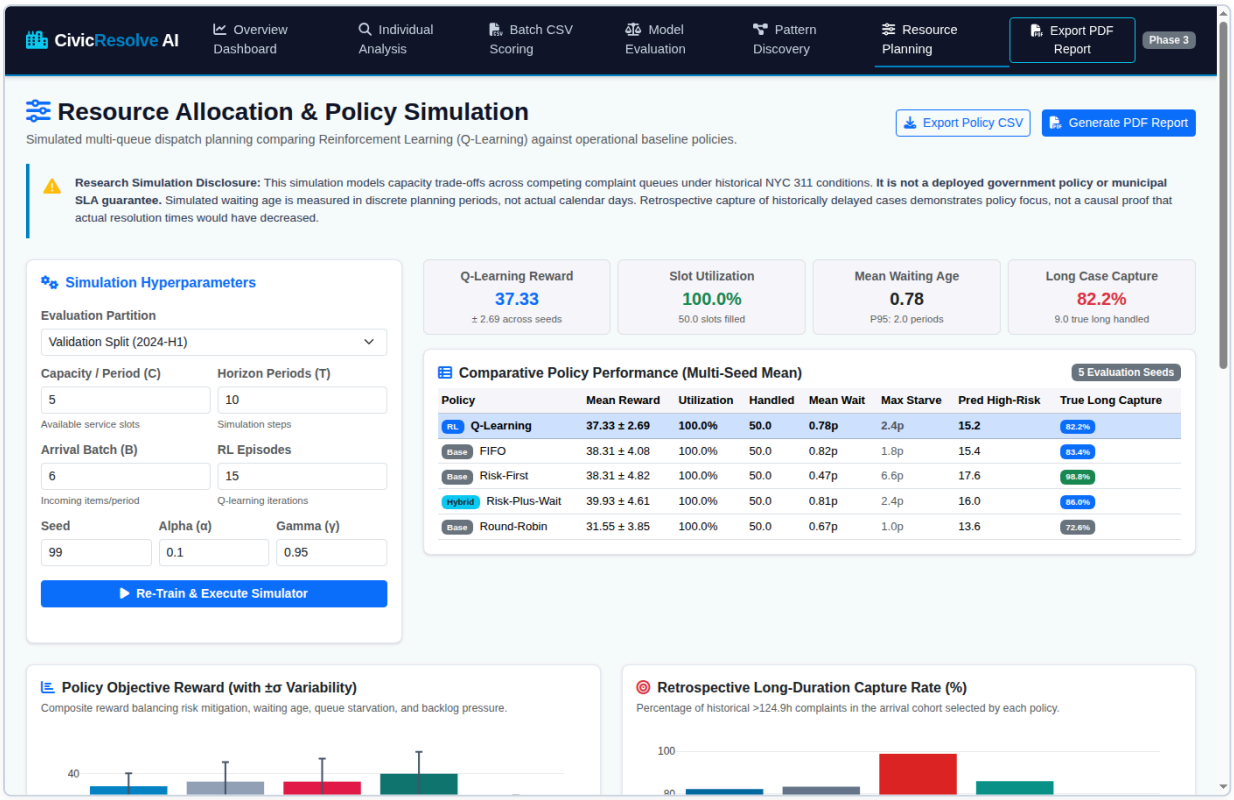


Figure 7.6: Resource Planning Simulator (`/resource-planning`) showing multi-policy evaluation and Q-learning metrics.

CivicResolve AI

Municipal Operational Intelligence & Delay Risk Diagnostic

*Generated on **2026-10-24 03:36:54** | Filter Scope: Borough: **ALL** | Category: **ALL** | Window: **2023-01-02 to 2024-12-30***

1. Executive Summary

This diagnostic synthesizes **10,000** municipal service requests within the selected filter scope. Of these records, **9,778** have been resolved, and **222** remain active or in-progress. The historical median resolution duration across eligible closed cases is **8.7 hours** (0.4 days). A total of **2,231** cases (22.8%) exceeded the project-defined long-duration threshold of **124.925 hours** (~5.2 days).

Total Intake Records	10,000	**Median Duration (Eligible)**	8.7h (0.4d)
Closed (Valid Timestamp)	9,778	**Unresolved / Unclosed Backlog**	222 (active + null date)
Service-Eligible Closed	9,777	**Exceeding 124.9h Benchmark**	2,231 (22.8%)

2. Top Complaint Categories & Borough Distribution

Historical demand patterns demonstrate pronounced heterogeneity by complaint category and municipal jurisdiction. Top complaint types represent the primary service volume drivers:

Complaint Category	Volume	Share of Filter	Median Duration
Illegal Parking	1,466	14.7%	1.3h
Noise - Residential	924	9.2%	0.8h
HEAT/HOT WATER	734	7.3%	37.8h
Blocked Driveway	508	5.1%	1.6h
Noise - Street/Sidewalk	421	4.2%	0.6h

Borough	Complaints	Median Duration	Delay Rate (>124.9h)
BROOKLYN	3,037	8.5h	23.8%
QUEENS	2,370	4.9h	16.0%
MANHATTAN	2,118	13.3h	28.3%
BRONX	2,084	14.7h	24.2%
STATEN ISLAND	383	15.3h	19.1%

Source: NYC 311 Sample (2023–2024) | Strictly Evaluated Research Layer Page 1 of 3

Figure 7.7: Multi-page Diagnostic PDF Report (`/report/download`) generated via ReportLab.

Chapter 8: Conclusion, Limitations & Future Work

8.1 Summary of Contributions

CivicResolve AI demonstrates that municipal complaint management can be substantially enhanced through leakage-free machine learning, unsupervised pattern segmentation, and reinforcement learning capacity planning. The project bridges the gap between theoretical modeling and public administration realities.

8.2 Defensible Academic Limitations

1. **No Causal Guarantee:** Feature associations do not prove physical causality; routing adjustments cannot guarantee real-world duration drops without operational field trials.
2. **Context Specificity:** Models trained on New York City data cannot be directly deployed in Indian municipalities (e.g., PCMC) due to distinct administrative hierarchies, complaint typologies, and regional reporting channels.
3. **Tabular Discretization:** The Q-learning agent uses coarse state bins (1,764 combinations), which limits micro-item optimization compared to continuous heuristic sorting.

8.3 Future Enhancements

- Fine-tuning local language (Marathi/Hindi) intake NLP for Indian municipal corporations.
- Deep Q-Networks (DQN) or Proximal Policy Optimization (PPO) for continuous state spaces.
- Real-time GPS vehicle routing integration for municipal field crews.

Chapter 9: References

1. **City of New York (2024).**
   311 Service Requests from 2010 to Present.
   NYC Open Data. https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-Present/erm2-nwe9
2. **Pedregosa, F., et al. (2011).**
   Scikit-learn: Machine Learning in Python.
   Journal of Machine Learning Research, 12, 2825–2830. https://scikit-learn.org/
3. **Sutton, R. S., & Barto, A. G. (2018).**
   Reinforcement Learning: An Introduction.
   MIT Press, Cambridge, MA. http://incompleteideas.net/book/the-book-2nd.html
4. **Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022).**
   Why do tree-based models still outperform deep learning on tabular data?
   Advances in Neural Information Processing Systems (NeurIPS). https://arxiv.org/abs/2207.08815
5. **Rousseeuw, P. J. (1987).**
   Silhouettes: A graphical aid to the interpretation and validation of cluster analysis.
   Journal of Computational and Applied Mathematics, 20, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7
6. **ReportLab Europe Ltd. (2024).**
   ReportLab PDF Generation User Guide.
   https://www.reportlab.com/documentation/
7. **Flask Community (2024).**
   Pallets: Flask Web Development.
   https://flask.palletsprojects.com/